

LAB 1 — PL/pgSQL, Regex, ROLLUP, Dates

Task 1: Select rows from t1 where year >= 2010

```
SELECT * FROM t1
WHERE CAST(
    REGEXP_REPLACE(data, '[^0-9]', '', 'g')
    AS INT) >= 2010;
-- Or with substring:
SELECT * FROM t1
WHERE (REGEXP_MATCH(data, '\d{4}')) [1]::INT
    >= 2010;
```

Task 2: PL/pgSQL — extract year from t1 & print

```
DO $$
DECLARE
    rec RECORD;
    yr TEXT;
BEGIN
    FOR rec IN SELECT data FROM t1 LOOP
        yr := (REGEXP_MATCH(rec.data, '\d{4}')) [1];
        IF yr IS NOT NULL THEN
            RAISE NOTICE 'Year: %', yr;
        END IF;
    END LOOP;
END;$$;
```

Task 3: Depts where total sales > 1200

```
SELECT department,
    SUM(sales) AS total_sales
FROM order_details
GROUP BY department
HAVING SUM(sales) > 1200;
```

Task 4: PL/pgSQL — print depts total sales > 1000

```
DO $$
DECLARE rec RECORD;
BEGIN
    FOR rec IN
        SELECT department, SUM(sales) AS tot
        FROM order_details
        GROUP BY department
        HAVING SUM(sales) > 1000
    LOOP
        RAISE NOTICE 'Dept: %, Sales: %',
            rec.department, rec.tot;
    END LOOP;
END;$$;
```

Task 5: ROLLUP on order_details

```
SELECT department,
    SUM(sales) AS total_sales
FROM order_details
GROUP BY ROLLUP(department)
ORDER BY department;
```

Task 6: PL/pgSQL ROLLUP — print subtotals

```
DO $$
DECLARE rec RECORD;
BEGIN
    FOR rec IN
        SELECT department,
            SUM(sales) AS tot
        FROM order_details
        GROUP BY ROLLUP(department)
    LOOP
        RAISE NOTICE 'Dept: %, Total: %',
            COALESCE(rec.department, 'GRAND'),
            rec.tot;
    END LOOP;
END;$$;
```

Task 7: CUBE — employees by dept & designation

```
SELECT department, designation,
    COUNT(*) AS total
FROM employees
GROUP BY CUBE(department, designation)
ORDER BY department, designation;
```

Task 8: Format hire dates to DD-MM-YYYY

```
SELECT employee_id,
    TO_CHAR(hire_date, 'DD-MM-YYYY')
    AS formatted_date
FROM employees;
```

Task 9: PL/pgSQL — diff in years between earliest & latest hire

```
DO $$
DECLARE
    min_dt DATE; max_dt DATE;
    diff_yr INT;
BEGIN
    SELECT MIN(hire_date), MAX(hire_date)
    INTO min_dt, max_dt FROM employees;
    diff_yr := EXTRACT(YEAR FROM max_dt)
        - EXTRACT(YEAR FROM min_dt);
    RAISE NOTICE 'Year difference: %', diff_yr;
END;$$;
```

LAB 2 — Cursors, Triggers

Task 1: Implicit cursor — salary update, show rows affected

```
DO $$
DECLARE rows_affected INT := 0; n INT;
BEGIN
    UPDATE employee
    SET salary = salary * 1.10
    WHERE designation = 'manager'
        AND salary < 30000;
    GET DIAGNOSTICS n = ROW_COUNT;
    rows_affected := rows_affected + n;

    UPDATE employee
    SET salary = salary * 0.90
    WHERE designation = 'assistant manager'
        AND salary > 20000;
    GET DIAGNOSTICS n = ROW_COUNT;
    rows_affected := rows_affected + n;

    RAISE NOTICE '% row(s) affected', rows_affected;
END;$$;
```

Task 2: Loan scheme function with cursor

```
CREATE OR REPLACE FUNCTION
    get_loan_scheme(p_uid INT)
RETURNS INT LANGUAGE plpgsql AS $$
DECLARE
    v_total NUMERIC;
    v_scheme INT := 0;
    rec RECORD;
    cur CURSOR FOR
        SELECT scheme, min_trans
        FROM loan_type
        ORDER BY min_trans DESC;
BEGIN
    SELECT SUM(amount) INTO v_total
    FROM transactions WHERE user_id = p_uid;

    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        IF v_total >= rec.min_trans THEN
            v_scheme := rec.scheme;
            EXIT;
        END IF;
    END LOOP;
    END LOOP;
    RETURN v_scheme;
END;$$;
-- Usage:
SELECT get_loan_scheme(101);
```

Task 3.1: Trigger — init bill to 0 on new customer

```
CREATE OR REPLACE FUNCTION
    trg_init_bill()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO bill(ssn, month, year, amount)
    VALUES(NEW.ssn,
```

```
    EXTRACT(MONTH FROM CURRENT_DATE),
    EXTRACT(YEAR FROM CURRENT_DATE), 0);
    RETURN NEW;
END;$$;
```

```
CREATE TRIGGER trg_new_customer
AFTER INSERT ON customer
FOR EACH ROW
EXECUTE FUNCTION trg_init_bill();
```

Task 3.2: Trigger — update bill after phone call

```
CREATE OR REPLACE FUNCTION
    trg_update_bill()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_fee NUMERIC;
    v_pps NUMERIC;
    v_plan TEXT;
BEGIN
    SELECT c.plan INTO v_plan
    FROM customer c WHERE c.ssn = NEW.ssn;

    SELECT connection_fee, price_per_second
    INTO v_fee, v_pps
    FROM pricingplan WHERE code = v_plan;

    UPDATE bill
    SET amount = amount + v_fee
        + (v_pps * NEW.seconds)
    WHERE ssn = NEW.ssn
        AND month = EXTRACT(MONTH FROM NEW.date)
        AND year = EXTRACT(YEAR FROM NEW.date);
    RETURN NEW;
END;$$;
```

```
CREATE TRIGGER trg_after_call
AFTER INSERT ON phonecall
FOR EACH ROW
EXECUTE FUNCTION trg_update_bill();
```

Task 4.1: Gen_ID function + auto-insert trigger

```
CREATE SEQUENCE student_seq START 1;

CREATE OR REPLACE FUNCTION
    gen_id(p_date DATE, p_dept CHAR,
        p_prog CHAR, p_sec CHAR)
RETURNS VARCHAR LANGUAGE plpgsql AS $$
DECLARE
    yy TEXT; seq TEXT;
BEGIN
    yy := TO_CHAR(p_date, 'YY');
    seq := LPAD(nextval('student_seq')::TEXT, 2, '0');
    RETURN yy || '00' || p_dept
        || p_prog || p_sec || seq;
END;$$;

CREATE OR REPLACE FUNCTION trg_gen_id()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.id := gen_id(NEW.date_of_admission,
        NEW.department,
        NEW.program, NEW.section);
    RETURN NEW;
END;$$;

CREATE TRIGGER trg_student_id
BEFORE INSERT ON student
FOR EACH ROW
EXECUTE FUNCTION trg_gen_id();

-- Insert without ID:
INSERT INTO student
(date_of_admission, department,
    program, section)
VALUES ('2026-01-01', '13', '2', '1');
```

Task 4.2: Explicit cursor — update account balances with interest

```
CREATE OR REPLACE PROCEDURE
    apply_interest()
LANGUAGE plpgsql AS $$
DECLARE
    cur CURSOR FOR
        SELECT a.id, a.balance,
```

```
        a.last_date_of_interest,
        ap.interest_rate, ap.gp
    FROM accounts a
    JOIN account_properties ap
    ON a.acc_code = ap.id;
    rec cur%ROWTYPE;
    days_diff INT;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        days_diff :=
            CURRENT_DATE - rec.last_date_of_interest;

        IF (rec.gp=1 AND days_diff>=1) OR
            (rec.gp=2 AND days_diff>=30) OR
            (rec.gp=3 AND days_diff>=365) THEN
            UPDATE accounts
            SET balance = balance
                + balance * rec.interest_rate
                / 100,
                last_date_of_interest = CURRENT_DATE
            WHERE id = rec.id;
        END IF;
    END LOOP;
    CLOSE cur;
END;$$;
CALL apply_interest();
```

LAB 3 — Cursor/Trigger Tasks

Task 1: Explicit cursor — employees with salary > 50000

```
DO $$
DECLARE
    CURSOR c_emp IS
        SELECT emp_id, emp_name, salary
        FROM employees WHERE salary > 50000;
    rec c_emp%ROWTYPE;
BEGIN
    OPEN c_emp;
    LOOP
        FETCH c_emp INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'ID:% Name:% Sal:%',
            rec.emp_id, rec.emp_name, rec.salary;
    END LOOP;
    CLOSE c_emp;
END;$$;
```

Task 2: Cursor FOR UPDATE — raise salary in dept 20 by 10%

```
DO $$
DECLARE
    CURSOR c_dept20 IS
        SELECT emp_id, salary FROM employees
        WHERE dept_id = 20
        FOR UPDATE OF salary;
    rec c_dept20%ROWTYPE;
BEGIN
    OPEN c_dept20;
    LOOP
        FETCH c_dept20 INTO rec;
        EXIT WHEN NOT FOUND;
        UPDATE employees
        SET salary = salary * 1.10
        WHERE CURRENT OF c_dept20;
    END LOOP;
    CLOSE c_dept20;
END;$$;
```

Task 3: BEFORE INSERT trigger — set created_at = NOW()

```
CREATE OR REPLACE FUNCTION
    trg_set_created_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.created_at := NOW();
    RETURN NEW;
END;$$;

CREATE TRIGGER trg_students_created
BEFORE INSERT ON students
```

```
FOR EACH ROW
EXECUTE FUNCTION trg_set_created_at();
```

Task 4: AFTER INSERT trigger — log to emp_audit

```
CREATE OR REPLACE FUNCTION
trg_emp_audit()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO emp_audit
        (employee_id, action_time)
    VALUES (NEW.employee_id, NOW());
    RETURN NEW;
END;$$;
```

```
CREATE TRIGGER trig_after_emp_insert
AFTER INSERT ON employees
FOR EACH ROW
EXECUTE FUNCTION trg_emp_audit();
```

LAB 6 — JOINS & String Functions

Tasks 1-4: INNER / LEFT / RIGHT / FULL JOIN

```
-- T1: INNER JOIN      employees with dept name
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id;
```

```
-- T2: LEFT JOIN      all employees incl. no dept
SELECT e.emp_name,
       d.dept_name
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.dept_id;
```

```
-- T3: RIGHT JOIN     all depts incl. no employees
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
    ON e.dept_id = d.dept_id;
```

```
-- T4: FULL OUTER JOIN all rows both sides
SELECT e.emp_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
    ON e.dept_id = d.dept_id;
```

Task 5: CROSS JOIN — Cartesian product

```
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;
-- 5 employees x 4 depts = 20 rows
```

Tasks 6-11: String Function Tasks on CUSTOMERS

```
-- T6: Full name with space
SELECT CONCAT(first_name, ' ', last_name)
    AS full_name FROM customers;
```

```
-- T7: Proper case
SELECT INITCAP(first_name) AS first_name,
       INITCAP(last_name)  AS last_name
FROM customers;
```

```
-- T8: UPPER first_name, LOWER email
SELECT UPPER(first_name) AS first_name,
       LOWER(email)      AS email
FROM customers;
```

```
-- T9: Email length
SELECT email,
       LENGTH(TRIM(email)) AS email_length
FROM customers;
```

```
-- T10: Pad phone to 15 chars
SELECT phone,
       LPAD(phone, 15, '*') AS left_padded,
```

```
RPAD(phone, 15, '-') AS right_padded
FROM customers;
```

```
-- T11: First 4 chars + domain after 0
SELECT email,
       SUBSTR(email, 1, 4) AS first_4_chars,
       SUBSTR(email,
               POSITION('0' IN email)+1) AS domain
FROM customers;
```

LAB 5 — Recursive Queries & Window Functions

Task 1: Recursive — all direct & indirect task dependencies

```
WITH RECURSIVE task_deps
    (task_id, prereq_id) AS (
    -- Base: direct dependencies
    SELECT task_id, depends_on AS prereq_id
    FROM project_tasks
    WHERE depends_on IS NOT NULL
    UNION
    -- Recursive: indirect dependencies
    SELECT td.task_id, pt.depends_on
    FROM task_deps td
    JOIN project_tasks pt
        ON td.prereq_id = pt.task_id
    WHERE pt.depends_on IS NOT NULL
    )
SELECT * FROM task_deps
ORDER BY task_id, prereq_id;
```

Task 2: Rank employees by salary within dept (ties same rank)

```
SELECT emp_id, name, department, salary,
       RANK() OVER(
           PARTITION BY department
           ORDER BY salary DESC)
       AS dept_rank
FROM employee
ORDER BY department, dept_rank;
```

Task 3: Salary difference from previous employee (ordered by salary)

```
SELECT emp_id, name, salary,
       salary - LAG(salary) OVER(
           ORDER BY salary ASC)
       AS salary_difference
FROM employee
ORDER BY salary ASC;
```

Task 4: Org hierarchy — level, path, total subordinates

```
WITH RECURSIVE hierarchy AS (
    -- Base: top-level (no manager)
    SELECT emp_id, emp_name, manager_id,
           department,
           1 AS level,
           emp_name::TEXT AS path
    FROM employee WHERE manager_id IS NULL
    UNION ALL
    -- Recursive: direct reports
    SELECT e.emp_id, e.emp_name,
           e.manager_id, e.department,
           h.level + 1,
           h.path || ' ' > ' || e.emp_name
    FROM employee e
    JOIN hierarchy h
        ON e.manager_id = h.emp_id
    ),
sub_count AS (
    -- Count all subordinates per manager
    SELECT manager_id,
           COUNT(*) AS total_subs
    FROM employee
    WHERE manager_id IS NOT NULL
    GROUP BY manager_id
    ),
```

```
all_subs AS (
    SELECT h.emp_id,
           COUNT(h2.emp_id) AS total_subordinates
    FROM hierarchy h
    LEFT JOIN hierarchy h2
        ON h2.path LIKE h.path || ' >% '
    GROUP BY h.emp_id
    )
SELECT h.emp_id, h.emp_name,
       h.manager_id, h.level,
       a.total_subordinates
FROM hierarchy h
JOIN all_subs a USING(emp_id)
ORDER BY h.level, h.emp_id;
```

LAB 8 — ROLLUP/CUBE, Dates, Regex

Task 1.1: ROLLUP — revenue by region, country, city

```
SELECT region, country, city,
       SUM(quantity * unit_price
            * (1 - discount_percent/100))
       AS total_revenue
FROM sales_transactions
GROUP BY ROLLUP(region, country, city)
ORDER BY region, country, city;
```

Task 1.2: CUBE — revenue & avg discount by category, subcategory, region

```
SELECT category, subcategory, region,
       SUM(quantity * unit_price
            * (1 - discount_percent/100))
       AS total_revenue,
       ROUND(AVG(discount_percent), 2)
       AS avg_discount
FROM sales_transactions
GROUP BY CUBE(category, subcategory, region)
ORDER BY category, subcategory, region;
```

Task 2.1: Format transaction date as "Day, DD Month YYYY"

```
SELECT transaction_id,
       TO_CHAR(transaction_date,
               'Day, DD Month YYYY')
       AS formatted_date
FROM sales_transactions;
```

Task 2.2: Group by month-year, total transactions & revenue

```
SELECT
    TO_CHAR(transaction_date, 'Mon-YYYY')
    AS month_year,
    COUNT(*) AS total_transactions,
    ROUND(SUM(quantity * unit_price
                * (1 - discount_percent/100)), 2)
    AS total_revenue
FROM sales_transactions
GROUP BY TO_CHAR(transaction_date, 'Mon-YYYY')
ORDER BY MIN(transaction_date);
```

Task 2.3: Warranty expiry (+12 months) for Electronics

```
SELECT transaction_id, transaction_date,
       (transaction_date + INTERVAL '12 months')
       ::DATE AS warranty_expiry
FROM sales_transactions
WHERE category = 'Electronics';
```

Task 2.4: Days since each transaction (from 2024-03-01)

```
SELECT transaction_id, transaction_date,
       ('2024-03-01'::DATE - transaction_date)
       AS days_since_transaction
FROM sales_transactions;
```

Task 2.5: Transactions > 30 days before 2024-03-01

```
SELECT transaction_id, transaction_date
FROM sales_transactions
WHERE transaction_date
    < ('2024-03-01'::DATE - INTERVAL '30 days');
```

Task 3.1: Regex — find invalid emails

```
SELECT customer_id, full_name, email
FROM customer_data
WHERE email !~
    '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$';
-- Flags invalid: 'notenemail', '123456'
```

Task 3.2: Regex — extract title (Dr., Mr., Mrs., Ms.) from name

```
SELECT full_name,
       (REGEXP_MATCH(full_name,
                     '^(Mr\.|Mrs\.|Ms\.|Dr\.|Prof\.|s)')) [1]
       AS title,
       REGEXP_REPLACE(full_name,
                     '^(Mr\.|Mrs\.|Ms\.|Dr\.|Prof\.|s)', '')
       AS name_without_title
FROM customer_data;
```

LAB 9 — Normalization of Orders_Unnormalized

Task 1: Problems identified

Repeating groups: Product1/2/3 cols violate 1NF. **NULLs:** sparse product cols. **Partial deps:** customer data depends only on OrderID. **Transitive deps:** PaymentMethod → PaymentStatus.

Tasks 2-4: 1NF → 2NF → 3NF

```
-- 1NF: one product per row, composite PK
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    product_name VARCHAR(100),
    unit_price DECIMAL(10,2),
    quantity INT,
    PRIMARY KEY(order_id, product_id));
```

```
-- 2NF: split customer (depends only on order_id)
CREATE TABLE customers_nf (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    phone VARCHAR(20),
    address VARCHAR(255));
```

```
CREATE TABLE orders_2nf (
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT REFERENCES customers_nf);
```

```
CREATE TABLE products_nf (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    unit_price DECIMAL(10,2));
```

```
-- 3NF: remove transitive dep (payment)
CREATE TABLE payments_nf (
    order_id INT PRIMARY KEY
    REFERENCES orders_2nf,
    payment_method VARCHAR(50),
    payment_status VARCHAR(50));
```

Task 5: Anomalies in unnormalized design

Insert anomaly: Cannot add a new product without creating an order. **Update anomaly:** Changing CustomerPhone requires updating every row for that customer — inconsistency risk. **Delete anomaly:** Deleting the only order for a customer permanently loses all customer info.